

Capítulo 1 — GUÍA DE LECTURA

Capítulo 1 — GUÍA DE LECTURA.....	1
La web como plataforma	2
HTTP, el navegador y el documento, explicados en criollo.....	2
Antes de empezar: cómo usar esta guía.....	2
1.1 — De qué se trata esta clase	3
Qué dice	3
En criollo.....	3
1.2 — Por qué existe la web: el problema que vino a resolver	3
Qué dice	3
Las tres que ya existían, y por qué ninguna alcanzaba	4
Las cuatro decisiones que explican todo el módulo	4
Y después, cronología	5
1.3 — Qué pasa entre la barra de direcciones y la pantalla	5
Qué dice	5
En criollo: dos observaciones que valen por toda la lista	6
1.4 — Cómo está armado HTTP	7
1.4.1 — Petición y respuesta, y el servidor que no se acuerda de nada	7
Qué dice	7
Precisión de vocabulario, porque acá se confunden dos cosas	7
1.4.2 — Los métodos: seguridad e idempotencia	8
Qué dice	8
1.4.3 — Los códigos de estado	9
Tres pares que no hay que confundir	9
1.5 — Anatomía de una petición y de una respuesta	10
Los tres detalles que más se pasan por alto	11
1.6 — Anatomía de la URL	12
En criollo.....	13
1.7 — El navegador por dentro: del byte al píxel.....	14
1.7.1 — Del texto al DOM	14
Acá se ve la tercera decisión de diseño en acción, y es contraintuitiva	14
1.7.2 — De los estilos al píxel.....	14
1.8 — Anatomía del documento HTML	15
1.9 — HTML semántico: la etiqueta correcta no es decoración	16
1.9.1 — Por qué un div disfrazado de botón no es un botón	16
1.9.2 — Formularios	17
1.10 — Estudio de caso: leer la especificación de la API	18
1.11 — Herramientas de diagnóstico	19

1.12 — Seguridad y evolución del protocolo	20
Los encabezados que le ponen límites al propio navegador	20
Las tres versiones del protocolo, y lo que no cambió.....	21
1.13 — Verificación: el checklist honesto.....	21
1.14 — Los ocho errores frecuentes	21
1.15 — Las actividades, y qué busca cada una	23
1. Reconstruir el recorrido	23
2. Clasificar métodos.....	23
3. Diagnóstico de códigos.....	23
4. Documento semántico	23
5. Comparación DOM / HTML	23
6. Exploración: peticiones sin navegador.....	23
7. Exploración: el costo de la tolerancia	23
1.16 — Síntesis: las diez frases.....	24
1.17 — Qué leer, y en qué orden	24
Si lees una sola cosa.....	24
Si lees tres	24
Las fuentes normativas (para consultar, no para leer de corrido)	25
Cierre: las seis cosas que hay que recordar	25

La web como plataforma

HTTP, el navegador y el documento, explicados en criollo

Traducción didáctica del texto académico, sección por sección. Mismos conceptos, otro idioma.

Antes de empezar: cómo usar esta guía

Este documento no reemplaza al capítulo original: lo traduce. El texto académico está bien escrito, pero está escrito en el idioma de los papers —denso, comprimido, cada frase cargada—. Esta guía lo desarma y lo cuenta como se lo contarías a alguien en un café.

La regla es una sola: **no se pierde ni un concepto**. Si el original dice idempotencia, acá dice idempotencia. Lo que cambia es que además te explico qué es y por qué te importa.

Cada sección tiene tres partes:

- **Qué dice** — la idea del original, en dos o tres oraciones.
- **En criollo** — la explicación larga, con la analogía que la hace pegar.
- **Para el pizarrón** — la frase que te tenés que llevar.

LA IDEA MADRE DE TODO EL CAPÍTULO

Si te quedás con una sola frase de las cuarenta páginas, que sea esta:

La web no se diseñó de una vez: se acumuló durante treinta años sobre un puñado de decisiones tomadas cuando nadie sabía para qué se iba a usar.

Casi todas las rarezas que vas a encontrar en los capítulos que siguen —incluidas las de JavaScript— son compatibilidad hacia atrás con esas decisiones.

Por eso el capítulo empieza por el origen. No es anécdota histórica: **conocer el problema que una tecnología vino a resolver es la única forma de juzgar si una solución propuesta lo resuelve.**

1.1 — De qué se trata esta clase

Qué dice

El módulo de frontend abre por el único lugar posible: no por el código que se escribe, sino por la plataforma donde ese código se ejecuta. Antes de la primera línea de HTML hay una pregunta que casi nunca se formula y que gobierna todo lo demás: **¿qué pasa exactamente cuando alguien escribe una dirección y aparece una página?**

En criollo

Vos ya sabés programar. Variables, funciones, condicionales, estructuras de datos: eso lo trabajaste en Programación 1 y 2. Lo que no conocés es **el entorno donde ese conocimiento va a ejecutarse**, y ese entorno tiene reglas propias que no se deducen de saber programar.

Y no es cultura general. Fijate en tres consecuencias concretas:

Si no sabés esto...	...no vas a entender esto otro
Que el protocolo de la web no recuerda nada entre una petición y la siguiente	Por qué el TPI manda un token en cada llamada, y no una sola vez
Que el navegador jamás rechaza un documento mal formado	Por qué toda la validación que importa vive en el servidor
Que POST no es idempotente según la norma	Por qué existe la regla RN-F07 —la clave de idempotencia del checkout—, que sin este capítulo parece un capricho del enunciado y es la consecuencia directa de una decisión de 1996

PARA EL PIZARRÓN

El objetivo de la clase es concreto y medible: al terminar tenés que poder leer **la sección 6 del TPI** —la especificación de la API— y entender qué describe cada columna de esas tablas, **sin haber escrito todavía una línea de código de cliente.**

No aprendés HTTP para saber HTTP. Lo aprendés para poder leer la consigna.

1.2 — Por qué existe la web: el problema que vino a resolver

Qué dice

A fines de los ochenta, en el CERN, el problema no era la falta de información en red: era lo contrario. Miles de investigadores, decenas de sistemas incompatibles y rotación permanente de personal. La documentación de cada experimento vivía en el sistema que su autor hubiera elegido, en el formato que ese sistema impusiera, accesible sólo para quien conociera el procedimiento exacto. Cuando un investigador se iba, su documentación quedaba huérfana.

Tim Berners-Lee lo formuló con precisión en marzo de 1989, en una propuesta titulada *Information Management: A Proposal*: **la información existía, pero recuperarla exigía saber de antemano dónde estaba y cómo se accedía a ese lugar en particular.**

Las tres que ya existían, y por qué ninguna alcanzaba

Esto vale la pena mirarlo con cuidado, porque **cada una falló por una razón distinta**, y de esas tres razones salieron las tres decisiones de diseño de la web.

Herramienta	Qué hacía bien	Por qué no alcanzaba
FTP · transferencia de archivos	Traía un documento de una máquina remota	Había que saber el nombre de la máquina, la ruta exacta y con frecuencia unas credenciales. Y un documento traído por FTP era un archivo suelto: no había forma de que señalara a otro documento en otra máquina. El problema no era transferir: era relacionar
Gopher · Minnesota, 1991	Organizaba: cada servidor era un árbol de menús navegable. Por un tiempo creció más rápido que la web	Modelo jerárquico y cerrado: un elemento de un menú no podía apuntar al interior de otro servidor como si fuera parte del mismo tejido. Y en 1993 la universidad anunció que cobraría licencias — justo cuando la alternativa se liberaba
Hipertexto · Memex (1945), Xanadu (1960), HyperCard (1987)	Vinculaba documentos con capacidades que la web todavía hoy no tiene: enlaces bidireccionales, control de versiones, integridad referencial garantizada	Ninguno escaló a una red mundial, y por la misma razón los tres: garantizar que ningún enlace se rompa exige coordinación central, y la coordinación central no escala . Si el sistema promete integridad, alguien tiene que saber qué apunta a qué — y ese alguien es el cuello de botella de toda la red

Las cuatro decisiones que explican todo el módulo

De ese diagnóstico salieron cuatro decisiones. El resto del capítulo —y buena parte del resto del módulo— consiste en descubrir sus consecuencias.

Decisión	Qué gana	Qué cuesta
1. Identificadores universales. Cualquier recurso se nombra con una cadena única y global, sin catálogo central ni registro previo	Un documento puede señalar a otro sin pedirle permiso a nadie	Nada, y por eso es la que sostiene a las otras tres

Decisión	Qué gana	Qué cuesta
2. Un protocolo sin estado. El servidor no recuerda nada entre una petición y la siguiente	Puede atender millones de peticiones sin guardar contexto de ninguna — y por lo tanto replicarse en tantas máquinas como haga falta	La sesión hay que reconstruirla en cada petición. De ahí el <code>Authorization</code> con el token en casi todos los endpoints del TPI
3. Tolerancia al error de formato. El navegador nunca rechaza un documento mal formado: hace el mejor esfuerzo por mostrarlo igual	La web pudo crecer con millones de autores sin formación técnica. Fue deliberado	El HTML no valida nada. Ninguna comprobación hecha en el cliente es una garantía. La validación real vive en el servidor — en el TPI, los esquemas de la sección 7
4. Enlaces unidireccionales. Un documento apunta a otro sin que el apuntado lo sepa y sin que nadie verifique que el destino existe	Es exactamente lo que Xanadu se negaba a aceptar, y es la razón por la que la web escaló y Xanadu no	Los enlaces rotos. Se pagó a conciencia

Y después, cronología

En 1990 Berners-Lee implementó en una estación NeXT el primer servidor y el primer navegador — que además era editor, un detalle que se perdió en el camino—. El 25 de diciembre de ese año, la primera comunicación exitosa entre cliente y servidor. El 6 de agosto de 1991, el anuncio público en el grupo de noticias `alt.hypertext`. Y el 30 de abril de 1993, el CERN liberó el software al dominio público, sin regalías: la decisión que Minnesota no tomó.

PARA EL PIZARRÓN: la pregunta que hay que aprender a hacer

Fijate en algo que va a volver una y otra vez en este módulo: **las tres tecnologías que fracasaron eran mejores que la que ganó**. FTP transfería bien. Gopher organizaba bien. Xanadu vinculaba muchísimo mejor.

La web ganó porque resignó garantías a cambio de no necesitar coordinación.

Y esa es la forma de pensar que te va a servir cuando le pidas algo a un agente de IA: no preguntes solamente si una solución funciona, **preguntá qué resignó para funcionar**. Siempre resignó algo. Si no lo ves, es que todavía no lo entendiste.

1.3 — Qué pasa entre la barra de direcciones y la pantalla

Qué dice

Antes de formalizar el protocolo conviene tener el recorrido completo a la vista. Lo que sigue es lo que pasa cuando alguien escribe `https://foodstore.example/productos` y presiona Enter. Ocho pasos, y cada uno se desarrolla más adelante.

- Resolución del nombre.** El nombre `foodstore.example` no sirve para establecer una conexión: hace falta una dirección de red. El sistema operativo consulta al resolver configurado, que devuelve una dirección IP. Cuando este tramo falla, **el servidor no registra absolutamente nada**, porque la petición nunca llegó a emitirse.
- Conexión.** Con la dirección en mano se abre una conexión TCP contra el puerto correspondiente —443 para HTTPS— y sobre ella se negocia el cifrado TLS (*sección 1.12*).

3. **Petición.** El navegador emite una petición HTTP: una línea con el método y la ruta, un conjunto de encabezados y, si corresponde, un cuerpo (*sección 1.5*).
4. **Respuesta.** El servidor devuelve un código de estado, sus propios encabezados y el cuerpo de la respuesta (*secciones 1.4.3 y 1.5*).
5. **Parseo.** El navegador lee el HTML recibido y construye el **DOM**, la representación en memoria del documento (*sección 1.7.1*). Este es el objeto que todo el resto del módulo va a manipular.
6. **Recursos subordinados.** Durante el parseo el navegador descubre referencias a hojas de estilo, imágenes y scripts, y emite una petición nueva por cada una.
7. **Estilo, disposición y pintado.** El navegador combina el DOM con las reglas de estilo, calcula la geometría de cada elemento y dibuja los píxeles (*sección 1.7.2*).
8. **Ejecución de scripts.** El código JavaScript se ejecuta y puede modificar el DOM, lo que obliga a rehacer parte del trabajo anterior. Es el tema del Capítulo 4.

En criollo: dos observaciones que valen por toda la lista

La primera. Los pasos 1 a 4 **no son responsabilidad del frontend**, y sin embargo son donde se originan la mayoría de los errores que el frontend tiene que manejar. Un desarrollador de cliente que no distingue un fallo de resolución de nombre de un fallo de conexión, ni un error de red de un error de aplicación, va a escribir manejo de errores inútil: un `catch` que dice "hubo un problema" y no ayuda a nadie.

La segunda. El paso 5 produce el objeto sobre el que trabaja todo este módulo. El HTML que viaja por la red **es texto**; el DOM **es una estructura de datos en memoria**. Confundirlos es el error conceptual más común de quien empieza, y el Capítulo 4 está dedicado íntegramente a esa distinción.

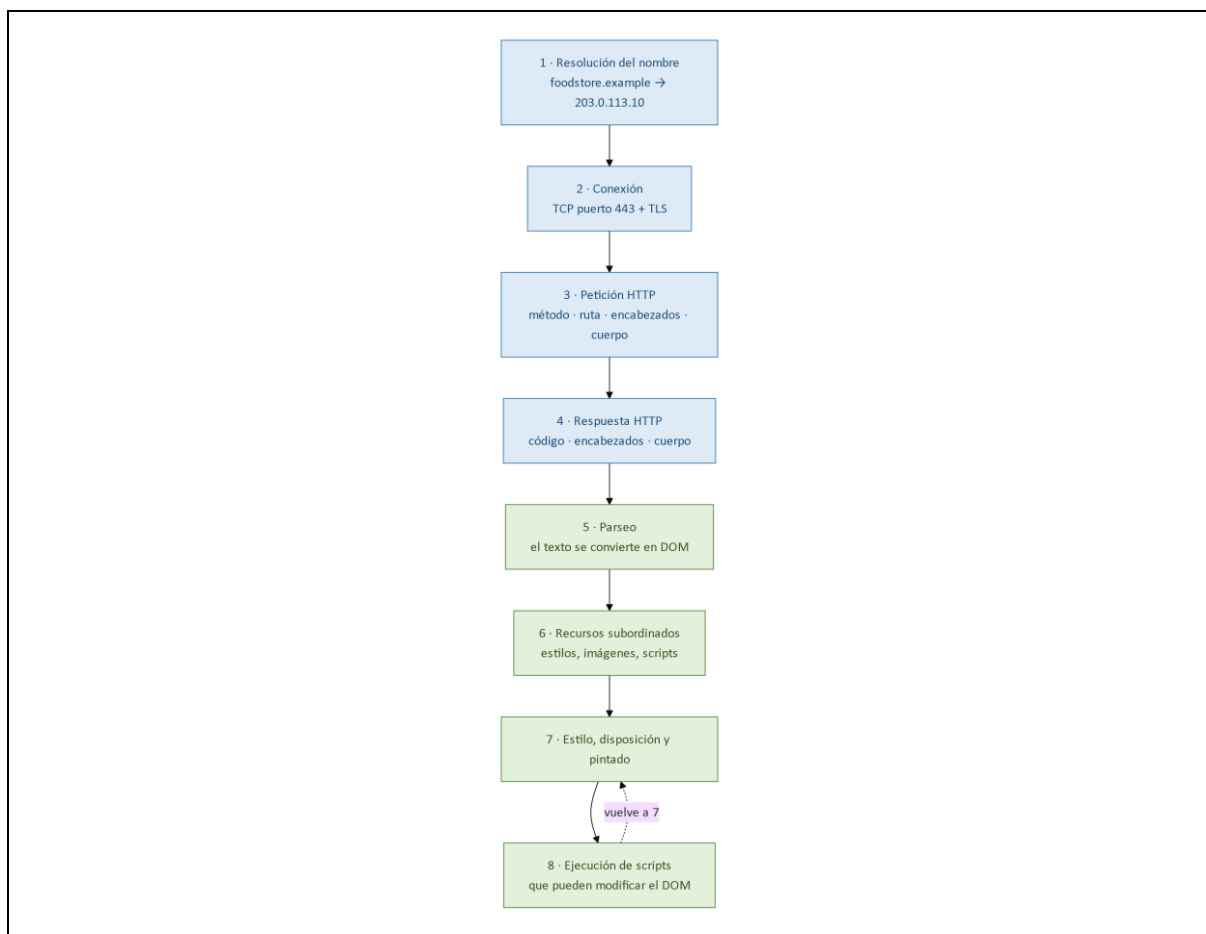


Figura 1.1. El recorrido de una petición

Un detalle del paso 6 que sorprende a todos

Una página no es una descarga: son decenas.

Vos escribís una dirección, pero mientras el navegador parsea el HTML va descubriendo hojas de estilo, imágenes, tipografías y scripts — y por cada uno emite **una petición nueva**.

Abrí el panel de red de cualquier sitio y contá. Casi siempre es un número mucho más grande del que esperabas.

1.4 — Cómo está armado HTTP

1.4.1 — Petición y respuesta, y el servidor que no se acuerda de nada

Qué dice

HTTP es un protocolo de petición y respuesta: el cliente pide, el servidor devuelve **exactamente una** respuesta, y el intercambio termina. **El servidor nunca inicia la conversación.**

Esa asimetría es constitutiva y tiene una consecuencia enorme, que el TPI enfrenta de lleno en su sección 11: si el servidor necesita avisarle algo al cliente —que un pedido cambió de estado, por ejemplo— **no puede simplemente mandárselo**. Hace falta un mecanismo adicional, y el Capítulo 5 estudia el que el TPI eligió.

La segunda característica es la que más consecuencias tiene: **HTTP no tiene estado**. El servidor no guarda nada sobre peticiones anteriores. Cada petición llega como si fuera la primera.

Precisión de vocabulario, porque acá se confunden dos cosas

Que **el protocolo** no tenga estado no significa que **la aplicación** no lo tenga. Significa que el estado no vive en el protocolo. La aplicación guarda usuarios, pedidos y stock en una base de datos, y eso es estado del bueno. Lo que el protocolo no hace es **asociar automáticamente una petición con la anterior**.

Esa asociación hay que construirla, y siempre se construye igual: **la petición lleva consigo todo lo que hace falta para reconstruir el contexto**. En el TPI eso es un token firmado que viaja en el encabezado Authorization con el esquema Bearer, y que el servidor verifica en cada petición sin consultar ninguna tabla de sesiones. La sección 5.1 del TPI lo dice con todas las letras: el backend no almacena sesiones ni listas de tokens.

PARA ENTENDER: la lista de invitados y la pulsera

Esto explica algo que molesta y parece redundante: **¿por qué tengo que mandar el token en cada llamada? ¿No sería más cómodo mandarlo una vez y que el servidor se acuerde?**

Pensalo como la entrada a un evento. Hay dos formas de controlar quién pasa:

La lista de invitados (el servidor se acuerda). Alguien anota que entraste. Funciona con una puerta. Con veinte puertas, todas tienen que compartir la lista, y actualizarla en tiempo real.

La pulsera (el token). No hay lista: la pulsera está firmada y cualquier puerta puede verificar la firma sola. **La incomodidad es que tenés que mostrarla cada vez.**

La web eligió la pulsera. La incomodidad de repetir el token es **el precio de que el sistema pueda crecer agregando máquinas**. Con un servidor solo no se nota; con veinte atrás de un balanceador, la otra opción se cae.

Es un intercambio, no un descuido. Otra vez la pregunta de la sección 1.2: ¿qué resignó para funcionar?

1.4.2 — Los métodos: seguridad e idempotencia

Qué dice

El método declara **qué se pretende hacer** con el recurso. La norma vigente —la RFC 9110— define un conjunto acotado y, esto es lo importante, clasifica cada método según **dos propiedades que no son sinónimas y que se confunden todo el tiempo**.

Propiedad	Qué significa	Quiénes la cumplen
Seguro	No está destinado a modificar el estado del servidor. La palabra "destinado" no está de adorno: la norma no puede impedir que un servidor mal escrito borre algo al recibir un GET; lo que establece es que quien emite un GET tiene derecho a suponer que no está modificando nada . Consecuencia: los intermediarios pueden repetirlo y cachearlo libremente	GET, HEAD, OPTIONS
Idempotente	Ejecutarlo varias veces produce el mismo efecto que ejecutarlo una vez. Ojo: no quiere decir que la respuesta sea igual. Un DELETE repetido puede devolver 204 la primera vez y 404 la segunda. Lo que se mantiene igual es el estado resultante : el recurso está borrado en los dos casos	GET, HEAD, OPTIONS, PUT, DELETE

El ejemplo de siempre para no olvidarse: **el botón del ascensor es idempotente** —apretarlo cinco veces llama un ascensor, no cinco—. **El botón de comprar no lo es**.

Y acá aparece el que importa

POST no es seguro ni idempotente. Ninguna de las dos. Repetir un POST puede crear dos recursos donde debía haber uno.

Esa carencia funda una de las once reglas obligatorias del frontend del TPI. Mirá la situación: el cliente emite un POST para crear un pedido y **la respuesta no llega**. Queda en la peor posición posible: **no sabe si el pedido se creó o no**.

Si reintenta, puede duplicarlo — el usuario recibe dos veces la misma comida y un cargo doble. Si no reintenta, puede perderlo.

La norma no resuelve ese problema **porque no puede**: es inherente a una red poco confiable. Lo resuelve la aplicación, y siempre igual: el cliente genera un identificador único **antes** de emitir la petición, lo manda como encabezado, y el servidor lo usa para reconocer un reenvío.

Eso es exactamente **RN-F07**: generar la clave con `crypto.randomUUID()` al entrar al último paso del checkout y enviarla en el encabezado Idempotency-Key.

Guardá esta conexión: es el modelo de todo el módulo

RN-F07 **no es una ocurrencia del enunciado**. Es la respuesta de la aplicación a una propiedad que la norma le atribuye a POST desde 1996.

Cuando llegues al Capítulo 8 y la implementes, no la vas a estar copiando de la consigna: **vas a saber qué problema resuelve**. Y si un agente de IA te propone un checkout sin clave de idempotencia, vas a poder decirle exactamente qué le falta y por qué.

1.4.3 — Los códigos de estado

Toda respuesta empieza por un número de tres dígitos. **El primer dígito define la familia**, y cada familia contesta una pregunta distinta.

Familia	Significado	Qué informa realmente
1xx	Informativa	La petición se recibió y el proceso continúa
2xx	Éxito	Se recibió, se entendió y se procesó
3xx	Redirección	Hace falta una acción adicional para completarla
4xx	Error del cliente	La petición está mal. Repetirla igual no sirve
5xx	Error del servidor	La petición podía estar bien; el servidor falló

La distinción entre 4xx y 5xx es la que gobierna la lógica de reintento, y es también la que más se maltrata. Un **4xx** dice que la petición estaba mal: reintentarla sin cambios va a dar el mismo resultado y sólo consume recursos. Un **5xx** dice que el servidor no pudo, y ahí un reintento **sí** puede tener sentido, porque la causa puede ser transitoria.

Tres pares que no hay que confundir

El TPI los usa con precisión en su catálogo de la sección 14.1:

Par	La diferencia, en una frase	El error típico
401 vs 403	El 401 dice « no sé quién sos »: falta la credencial o no es válida. El 403 dice « sé quién sos y no te alcanza »: la credencial es válida pero el rol no autoriza esa operación	Mandar al usuario a la pantalla de login cuando el problema era de permisos. Ya está autenticado; volver a pedirle la contraseña no cambia nada y lo desconcierta
400 vs 422	El 400 es una petición malformada . El 422 es una petición bien formada cuyo contenido no pasó la validación semántica	Tratar los dos como "datos inválidos" y no poder decirle al usuario qué corregir
404 vs 410	El 404 dice no se encontró . El 410 dice existió y ya no está , de forma permanente	Seguir reintentando o mantener el enlace vivo ante un 410

Mención aparte para el **429**, que indica que el cliente superó un límite de peticiones. El TPI lo usa en su límite de intentos de autenticación —sección 4.4— y lo acompaña del encabezado **Retry-After**, que dice cuánto esperar. **Un cliente que ignora ese encabezado y reintenta de inmediato empeora exactamente el problema que el límite intenta contener.**

OJO ACÁ: un código de estado no es un cartel de error

Es **información de diagnóstico**. Cuando algo falla y ves un 401, no anotes «falló la petición»: anotá «**el servidor me dijo que no sabe quién soy**». Son cosas distintas y llevan a arreglos distintos.

Y prestá muchísima atención a esta otra diferencia, que es de las que cuestan una tarde entera:

Recibir un 500 significa que la petición **llegó**, el servidor la procesó y explotó adentro. Hay log del lado del servidor y lo podés buscar.

No recibir nada significa que la petición **nunca llegó**. No hay log de nada: el problema está en la red, en la resolución del nombre o en la conexión.

Confundir esos dos casos es buscar durante horas en el lugar equivocado.

1.5 — Anatomía de una petición y de una respuesta

Una petición HTTP es **texto con una estructura fija**. Conviene verla entera una vez, campo por campo, porque todo lo que el TPI especifica en su sección 6 es exactamente esto.

```
POST /api/v1/pedidos HTTP/1.1
Host: foodstore.example
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
Content-Type: application/json
Idempotency-Key: 6f4a2b18-9c3d-4e5a-8f01-2d7b6c9e4a13
Accept: application/json
Content-Length: 87
```

```
{"direccion_id": 42, "items": [{"producto_id": 7, "cantidad": 2}]}
```

La respuesta tiene la misma estructura, con una diferencia en la primera línea:

```
HTTP/1.1 201 Created
Content-Type: application/json
Location: /api/v1/pedidos/1043
Content-Length: 214
```

```
{"id": 1043, "estado": "pendiente", "total": "4750.00", ...}
```



Figura 1.2. Anatomía de una petición HTTP

Los tres detalles que más se pasan por alto

La línea en blanco no es formato: es el delimitador. Le dice al receptor dónde terminan los metadatos y dónde empiezan los datos. Los métodos sin cuerpo, como `GET`, terminan ahí.

La frase del código de estado es para las personas. En `HTTP/1.1 201 Created`, el "Created" no está garantizado por ninguna norma: ningún programa debe tomar decisiones leyéndolo. **El número sí está garantizado.** Y el encabezado `Location`, que acompaña a un 201, indica dónde quedó el recurso recién creado.

El total viaja como cadena de texto. Mirá el cuerpo de la respuesta: `"total": "4750.00"`, entre comillas, no como número. No es un descuido del ejemplo: es la regla **RN-F08** del TPI. El tipo numérico de JavaScript no puede representar exactamente ciertos valores decimales, y el dinero no admite ese error. El Capítulo 6 lo desarrolla; el módulo de backend lo ve desde el otro lado, cuando guarda el importe con un tipo decimal exacto.

EXPERIMENTO — hacelo hoy, en cualquier sitio

Abrió cualquier página y presioná **F12**. Andá a la pestaña **Network / Red** y recargá con la pestaña abierta.

1. **Uno.** Contá cuántas peticiones se emitieron. ¿Cuántas esperabas?
2. **Dos.** Hacé clic en la primera de todas, la del documento, y buscá los encabezados de petición y de respuesta. Encontrá los que acabás de leer acá.

3. **Tres.** Buscá una petición que haya devuelto un código que no sea 200 y anotá cuál es y de qué recurso se trata.

4. **Cuatro.** Ordená la lista por tamaño y fijate qué recurso pesa más.

Lo que estás mirando es **exactamente el texto de esta sección, capturado en vivo**. Esta pestaña va a ser tu herramienta principal durante los ocho capítulos.

1.6 — Anatomía de la URL

El identificador de recurso es la primera de las cuatro decisiones de la sección 1.2, y **la que sostiene a las otras tres**. Su sintaxis está normada por la RFC 3986 y tiene seis piezas.

`https://foodstore.example:443/api/v1/productos?categoria=3&pagina=2#resultados`

Componente	Valor en el ejemplo	Delimitador	Qué identifica
Esquema	https	termina en ://	El protocolo, y con él cómo interpretar todo el resto
Autoridad	foodstore.example	hasta :, /, ? o #	El servidor que aloja el recurso
Puerto	443	empieza en :	El punto de conexión; se omite si es el del esquema
Ruta	/api/v1/productos	empieza en /	El recurso dentro del servidor
Consulta	categoria=3&pagina=2	empieza en ?	Parámetros como pares <code>clave=valor</code>
Fragmento	resultados	empieza en #	Un punto dentro del recurso ya recibido

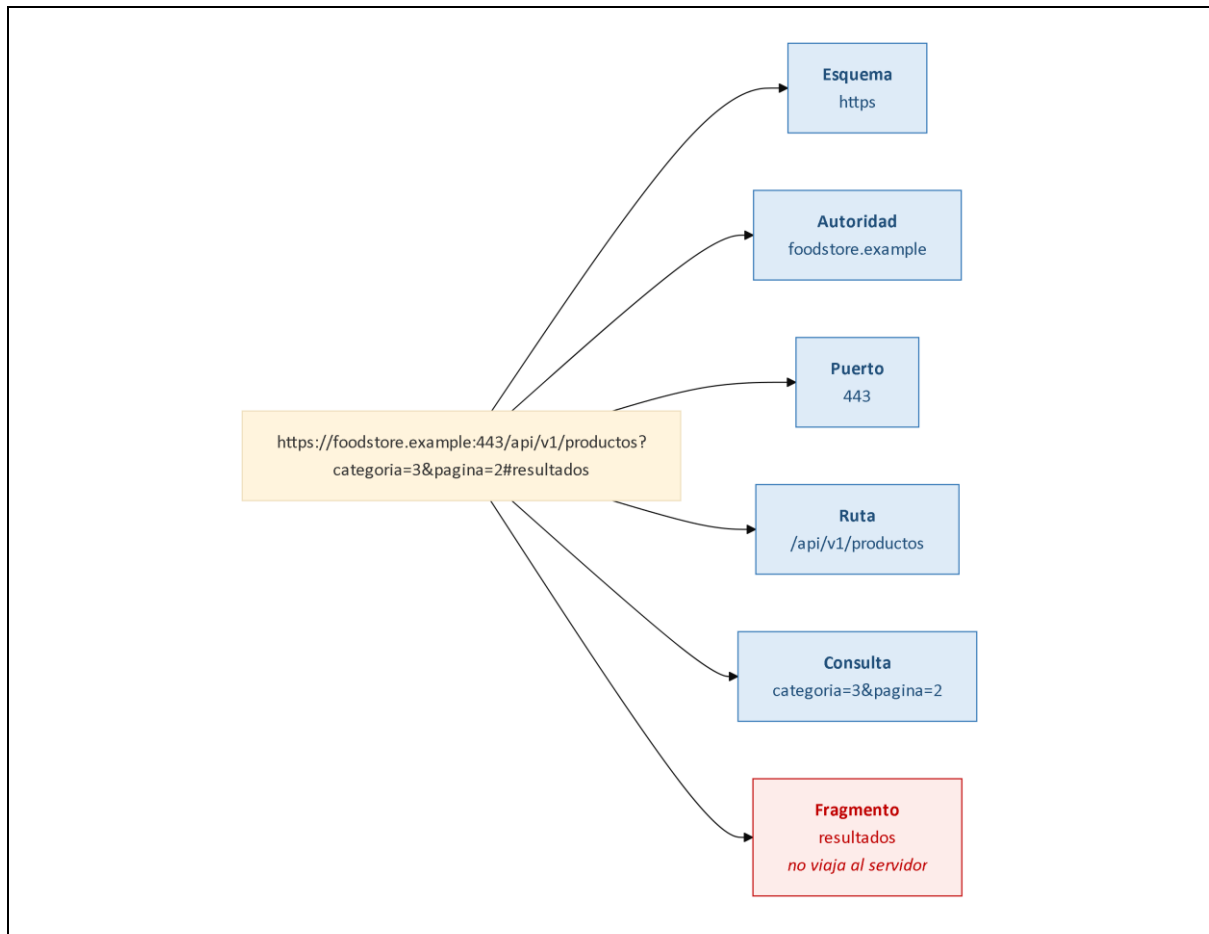


Figura 1.3. Anatomía de una URL

En criollo

El **esquema** indica el protocolo, y por lo tanto cómo interpretar todo lo que sigue. La **autoridad** nombra al servidor. El **puerto** casi siempre se omite porque cada esquema tiene el suyo: 443 para https, 80 para http. La **ruta** identifica el recurso dentro del servidor. La **cadena de consulta**, que empieza en `?`, transporta pares clave=valor separados por `&`; el TPI la usa para paginación y ordenamiento.

Y el **fragmento**, que empieza en `#`, merece párrafo propio porque tiene una propiedad que sorprende: **nunca se envía al servidor**. Es de uso exclusivo del cliente. El navegador lo usa para desplazarse hasta un elemento de la página **ya recibida**, y ninguna aplicación de servidor puede leerlo, porque jamás le llega.

La analogía: el fragmento es **la anotación que hacés en el margen de tu ejemplar del libro**. La editorial nunca se entera de que la hiciste.

OJO ACÁ: las dos caras del mismo detalle

Si mandás algo sensible después del `#`, el servidor no lo va a ver nunca. No es que llegue y se ignore. No llega. A veces eso te salva y a veces te desconcierta durante un rato largo buscando un parámetro que nunca salió del navegador.

Y al revés: todo lo que ponés en la cadena de consulta —después del `?`— sí viaja, y **queda escrito**: en los logs del servidor, en el historial del navegador y en el encabezado `Referer` de la navegación siguiente.

Nunca pongas ahí una contraseña ni un token. Esas cosas van **en el cuerpo de la petición o en un encabezado**, que no se registran de la misma manera.

1.7 — El navegador por dentro: del byte al píxel

1.7.1 — Del texto al DOM

Lo que llega por la red es **una secuencia de bytes**. Lo que el código de este módulo va a manipular es otra cosa: **una estructura de datos en memoria**. El proceso que convierte lo primero en lo segundo se llama **parseo**, y su resultado es el Document Object Model: el **DOM**.

El parser lee el texto, reconoce las etiquetas y construye un árbol de nodos. Cada elemento es un nodo, cada texto es un nodo, y la anidación del marcado se convierte en la relación padre-hijo del árbol.

Acá se ve la tercera decisión de diseño en acción, y es contraintuitiva

Si el documento tiene una etiqueta sin cerrar, o cerrada en el orden equivocado, o directamente inventada, **el parser no falla**. Aplica un conjunto de reglas de recuperación definidas en la norma y produce un árbol igual. El árbol puede no ser el que el autor esperaba, pero es un árbol válido, y la página se muestra.

La contracara es severa: un error de marcado no produce ningún mensaje de error. Simplemente aparece algo distinto de lo esperado, sin explicación. El navegador es ese amigo que te entiende igual aunque hables mal, y que nunca te corrige — por eso nunca te enterás de que hablaste mal.

El Capítulo 4 vuelve sobre esto: es una de las razones por las que conviene construir los nodos por programa en lugar de armar cadenas de texto con marcado adentro.

PARA ENTENDER: el HTML no es el DOM

Si agarrás esta distinción ahora, te ahorrarás meses de confusión.

El HTML es el plano en papel. El DOM es la casa construida. Si tirás una pared, la casa cambia — el plano no.

Dicho sin metáfora: el HTML es el texto que viajó por la red; el DOM es la estructura que el navegador construyó a partir de ese texto. **Dos cosas distintas, en dos momentos distintos.**

¿Por qué importa tanto? Porque a partir del Capítulo 4 vas a modificar el DOM con JavaScript, y cuando lo hagas **el HTML original no cambia**:

- **Ctrl+U** te muestra el texto que llegó, sin tus cambios.
- El panel **Elements** te muestra el DOM actual, con tus cambios.

Muchísima gente pierde tardes enteras porque mira uno y cree que está mirando el otro. Vos ya no.

1.7.2 — De los estilos al píxel

Con el DOM construido el navegador todavía no puede dibujar nada, porque no sabe cómo se ve cada nodo. Necesita las reglas de estilo, y con ellas arma una segunda estructura —el **CSSOM**—, que es el equivalente del DOM para las hojas de estilo. De la combinación de ambos sale el **árbol de render**, que contiene únicamente lo que se va a dibujar. Sobre él se ejecutan la **disposición** (calcular la geometría de cada caja) y el **pintado** (convertir esa geometría en píxeles).

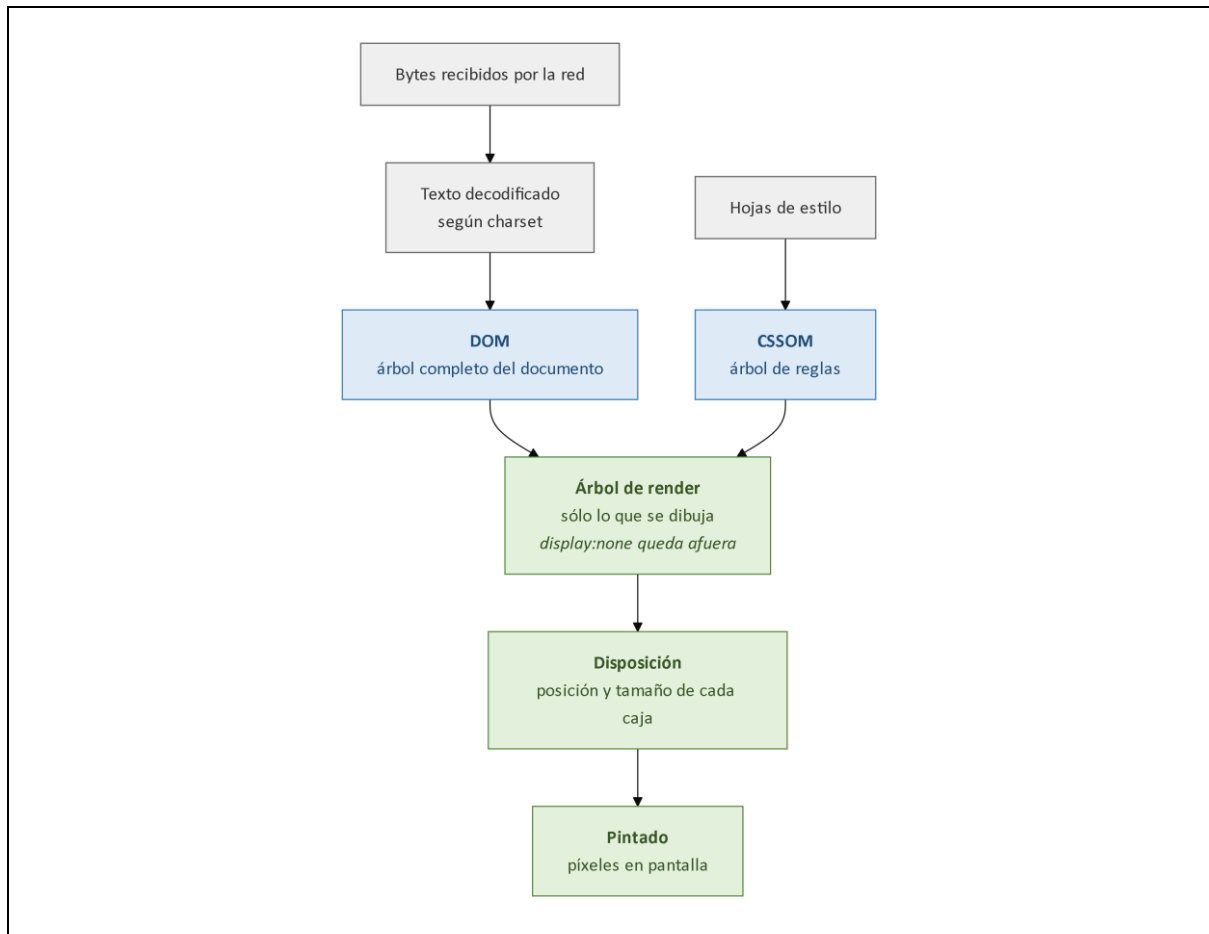


Figura 1.4. Del byte al píxel

Este recorrido explica dos comportamientos que de otra manera parecen arbitrarios.

Por qué las hojas de estilo bloquean el pintado. El navegador no dibuja hasta tener el CSSOM completo. Si dibujara antes, mostraría el documento sin estilos y después saltaría a la versión con estilos, produciendo un parpadeo. Prefiere esperar.

Por qué un script en medio del documento detiene el parseo. Un script puede modificar el DOM, incluso agregar contenido justo en el punto donde está. El parser no puede seguir construyendo un árbol que el script está por cambiar: se detiene, ejecuta el script y recién después continúa. De ahí salen los atributos `defer` y `async` que el Capítulo 3 estudia, y la costumbre de poner los scripts al final del documento.

1.8 — Anatomía del documento HTML

Un documento mínimo y correcto tiene esta forma. **Cada línea de la cabecera está por una razón concreta, y ninguna es ceremonial.**

```

<!DOCTYPE html>
<html lang="es">
  <head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Food Store – Catálogo</title>
    <link rel="stylesheet" href="/estilos.css">
  </head>
  <body>

```

```
<h1>Catálogo</h1>
<script type="module" src="/main.js"></script>
</body>
</html>
```

La línea	Qué hace de verdad	Si falta
<code><!DOCTYPE html></code>	No declara una versión: activa el modo estándar de renderizado. Va primero porque tiene que ser lo primero que el parser vea	El navegador entra en <i>quirks mode</i> : un modo de compatibilidad que reproduce el comportamiento de los navegadores de los noventa, con un modelo de caja distinto del actual
<code>lang="es"</code>	Declara el idioma del contenido. Lo usan los lectores de pantalla para elegir la pronunciación, los correctores y los buscadores	No rompe nada visible — y por eso se olvida siempre
<code><meta charset="utf-8"></code>	Declara la codificación de caracteres. Debe estar dentro de los primeros 1024 bytes	El navegador adivina. Cuando adivina mal aparecen los caracteres partidos que todo el mundo reconoce
<code><meta name="viewport"></code>	Le dice al navegador móvil que use el ancho real del dispositivo	Un teléfono simula una pantalla de escritorio de unos 980 píxeles y reduce todo: ningún diseño responsive funciona
<code><title></code>	El título del documento: pestaña, historial, favoritos y resultados de búsqueda	Es también lo primero que anuncia un lector de pantalla al cargar la página

Por qué lo del charset es circular, y por eso va tan arriba

Para leer el documento hay que saber **cómo están codificados sus caracteres...** y esa información está **dentro del propio documento**. Es una serpiente que se muerde la cola, y por eso la norma exige que la declaración esté en el primer kilobyte.

Si el navegador ya empezó a interpretar el documento con la codificación equivocada, cuando encuentre tu declaración **tiene que descartar todo lo que hizo y empezar de nuevo**. No es superstición: es trabajo tirado.

Y por si hace falta decirlo: **usá UTF-8 siempre**. Es la única codificación que cubre los acentos, la eñe y cualquier otro alfabeto sin sorpresas. Cualquier otra elección te va a dar problemas y no te va a dar ninguna ventaja.

1.9 — HTML semántico: la etiqueta correcta no es decoración

1.9.1 — Por qué un div disfrazado de botón no es un botón

Con suficiente CSS, un `<div>` puede verse **exactamente igual** que un `<button>`. Como se ven igual, es tentador concluir que da lo mismo cuál se use. **No da lo mismo, y la razón no es estética.**

El navegador construye, además del DOM, una segunda estructura derivada: **el árbol de accesibilidad**. Es la representación del documento que reciben las tecnologías de asistencia —

lectores de pantalla, navegación por teclado, software de control por voz —, y en ese árbol cada nodo tiene **un rol, un nombre y un estado**. Ese rol sale de la etiqueta.

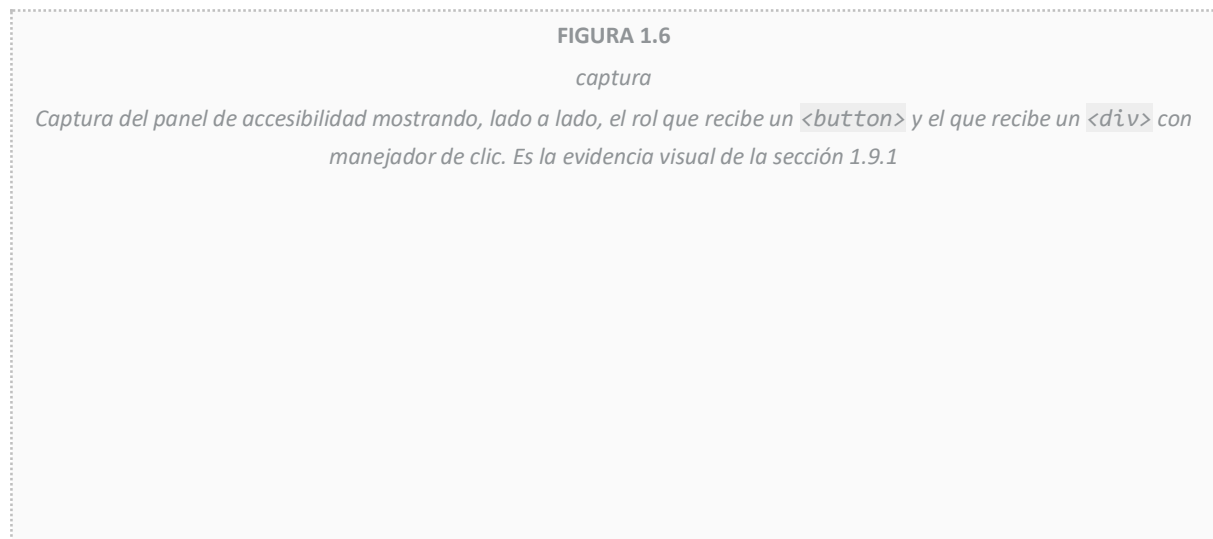


Figura 1.6. El árbol de accesibilidad

Es la diferencia entre una puerta y **una puerta pintada en la pared**. De lejos son idénticas. Una se abre.

De ahí sale una regla que conviene adoptar sin discutir: **usar el elemento cuyo significado corresponda a la función, y ajustar la apariencia con CSS**. Es menos trabajo y funciona mejor.

Los elementos estructurales siguen la misma lógica. `<header>`, `<nav>`, `<main>`, `<article>`, `<section>`, `<aside>` y `<footer>` no se ven distinto de un `<div>`, pero producen **puntos de referencia** en el árbol de accesibilidad. Un lector de pantalla puede saltar directamente al contenido principal si existe un `<main>`; si todo son `<div>`, el usuario tiene que recorrer el documento entero **cada vez que entra**.

1.9.2 — Formularios

Los formularios concentran **la mayor densidad de semántica del HTML**, y también la mayor cantidad de errores.

La pieza central es la asociación entre una etiqueta y su campo. **Escribir el texto al lado del campo no alcanza**: hay que vincularlos explícitamente, con el atributo `for` de la etiqueta apuntando al `id` del campo, o anidando el campo dentro de la etiqueta.

```
<label for="email">Correo electrónico</label>
<input type="email" id="email" name="email" required>
```

Esa asociación produce **tres efectos concretos**:

- El lector de pantalla **anuncia el nombre del campo** al enfocarlo, en lugar de decir simplemente "campo de texto".
- El clic sobre la etiqueta **lleva el foco al campo**, lo que agranda el área de interacción — importante en un teléfono.
- El mensaje de validación del navegador **puede nombrar el campo**.

Y el atributo `type` también hace más de lo que parece: determina **el teclado que aparece en un dispositivo móvil**, habilita la validación integrada del navegador y define qué sugerencias de autocompletado se ofrecen.

OJO ACÁ: lo más importante de la sección — la regla RN-F04**La validación del navegador es comodidad, no seguridad.**

El atributo `required` y el `type="email"` le avisan al usuario que se equivocó antes de mandar el formulario. Eso está muy bien y hay que ponerlo.

Pero cualquiera lo desactiva desde las herramientas de desarrollo en cinco segundos, o directamente manda la petición sin pasar por tu formulario. Ni siquiera hace falta ser hostil: alcanza con un script mal escrito.

El servidor tiene que validar todo de nuevo, **siempre, sin excepción**. En el TPI eso son los esquemas de la sección 7.

Si alguna vez pensás «esto ya lo validé en el frontend, no hace falta atrás», pará: **acabás de abrir un agujero**. Es la misma regla que el turno de backend estudia desde su lado — allá sus garantes son tres casos de prueba que le pegan al servidor salteándose tu interfaz.

1.10 — Estudio de caso: leer la especificación de la API

Todo lo anterior converge en un ejercicio concreto. La sección 6 del TPI describe **setenta endpoints** repartidos en once módulos, y cada uno se especifica con **una fila de tabla**. Esta es la primera fila del módulo de autenticación:

Método	Endpoint	Body / Params	Response	Auth
POST	/api/v1/auth/login	{ email, password }	200 TokenResponse	No — con límite

A esta altura del capítulo esa fila es **completamente legible**. Desarmémosla para comprobarlo:

La celda	Qué sabés de ella, y desde dónde
POST	Por la sección 1.4.2: no es seguro ni idempotente . Que un login sea POST y no GET no es convención arbitraria: las credenciales no pueden viajar en la ruta , porque la ruta queda registrada en logs e historial (sección 1.6)
/api/v1/auth/login	Es el componente de ruta del URI de la sección 1.6. El v1 es una decisión de diseño de la API: versionar en la ruta permite publicar una versión nueva sin romper los clientes de la anterior
{ email, password }	Es el cuerpo , que viaja después de la línea en blanco de la sección 1.5, en JSON declarado por Content-Type
200 TokenResponse	Un código de la familia 2xx (sección 1.4.3) más la forma del cuerpo de la respuesta, especificada en la sección 7 del TPI. Ese es el contrato que el Capítulo 6 va a traducir a tipos de TypeScript
No — con límite	El endpoint no exige autenticación —lógico: es el que la produce— pero está sujeto al límite de intentos de la sección 4.4, que responde con el 429 y el Retry-After de la sección 1.4.3

Una sola fila de tabla, y adentro estaban la semántica de los métodos, la anatomía del URI, la estructura de la petición, las familias de códigos y el manejo del límite. La sección 6 completa del TPI son setenta filas como esta.

PARA EL PIZARRÓN: este es el punto del capítulo

No aprendiste HTTP para saber HTTP. Lo aprendiste para poder leer la consigna.

Cuando en el Capítulo 6 le pidas a un agente de IA que te genere el cliente de la API, el agente va a producir código que compila y parece correcto. **Vos vas a tener que decidir si está bien.**

Y para decidir eso hace falta saber si ese POST lleva clave de idempotencia, si distingue un 401 de un 403, y si respeta el Retry-After de un 429.

El agente escribe rápido. Vos tenés que saber qué mirar. Eso es lo que estamos construyendo.

1.11 — Herramientas de diagnóstico

El navegador trae adentro todo el instrumental necesario para observar lo que describe este capítulo. Se abre con **F12**.

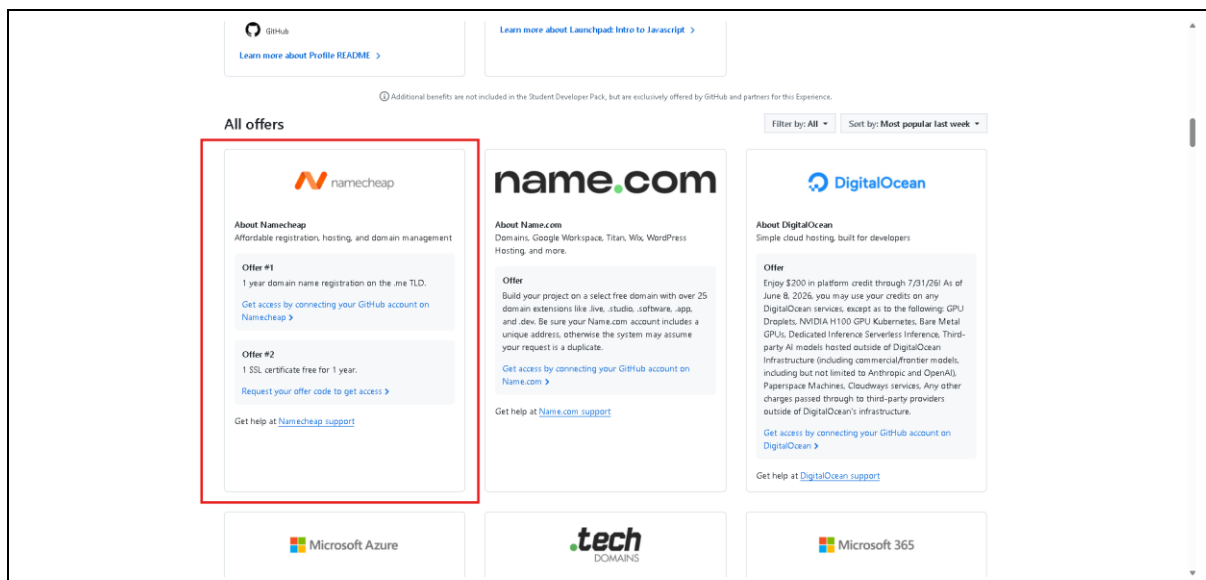


Figura 1.5. El panel Network

Pestaña	Qué muestra	Cuándo la usás
Network / Red	Todas las peticiones con método, código, tamaño y duración. Al seleccionar una, sus encabezados de petición y respuesta	Siempre. Conviene habilitar conservar el registro entre navegaciones : si no, una redirección borra justo la evidencia que querías ver
Elements / Elementos	El DOM actual , con las modificaciones que hayan hecho los scripts	Comparado con Ctrl+U —que pide al servidor el documento tal como llegó— es el ejercicio que separa los dos conceptos de la sección 1.7.1
Console / Consola	Errores de script, y permite ejecutar código contra la página	A partir del Capítulo 3

Pestaña	Qué muestra	Cuándo la usás
Accessibility	El árbol de la sección 1.9: rol, nombre y estado que las tecnologías de asistencia reciben de cada nodo	Cada vez que dudas si una etiqueta era la correcta

Fuera del navegador, `curl` emite peticiones desde la línea de comandos. Su **utilidad principal es aislar**: si una petición funciona con `curl` y no desde la página, el problema está en el código del cliente y no en el servidor.

```
curl -i https://foodstore.example/api/v1/productos

curl -i -X POST https://foodstore.example/api/v1/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"alguien@example.com","password":"secreto"}'
```

La opción `-i` incluye los encabezados de respuesta en la salida, que es justamente lo que interesa observar.

1.12 — Seguridad y evolución del protocolo

El protocolo descrito hasta acá tiene una carencia que en 1991 no se consideró problema y hoy es inaceptable: **HTTP viaja en texto plano**. Cualquiera con acceso al camino entre cliente y servidor puede leer el contenido — y también modificarlo sin que ninguna de las dos partes lo note.

HTTPS resuelve las dos cosas envolviendo HTTP en TLS. El protocolo de aplicación **no cambia en absoluto**: las mismas peticiones, los mismos encabezados, los mismos códigos. Lo que cambia es que el canal está cifrado y que el servidor presenta un certificado que acredita su identidad ante una autoridad reconocida.

El malentendido más caro sobre HTTPS

El certificado autentica al servidor, no al contenido.

HTTPS garantiza **con quién estás hablando** y que **nadie escuchó**. No garantiza que lo dicho sea verdad. **Un sitio con certificado válido puede mentirte con total comodidad.**

La analogía: es el sobre lacrado con el sello del remitente. Te asegura quién lo mandó y que nadie lo abrió en el camino. **No dice nada sobre si la carta dice la verdad.**

Los encabezados que le ponen límites al propio navegador

Sobre esa base se agregaron encabezados de respuesta que **instruyen al navegador a restringir su propio comportamiento**. Son una capa defensiva importante porque **actúan aunque la aplicación tenga errores**. El TPI los especifica en su sección 16.5:

Encabezado	Qué ordena	Contra qué protege
Strict-Transport-Security	Usar exclusivamente HTTPS con ese dominio durante un período determinado	Cierra la ventana de la primera visita por HTTP
Content-Security-Policy	Declara de qué orígenes se admite cargar scripts, estilos e imágenes	Es la defensa de fondo contra la ejecución de código inyectado . El Capítulo 4 la retoma al estudiar XSS

Encabezado	Qué ordena	Contra qué protege
X-Content-Type-Options: nosniff	Prohíbe al navegador ignorar el <code>Content-Type</code> declarado y deducir el tipo por el contenido	Un comportamiento heredado que se usó para hacer pasar scripts por imágenes

Las tres versiones del protocolo, y lo que no cambió

La evolución responde siempre al mismo problema. **HTTP/1.1** (1997) mantiene la conexión abierta entre peticiones pero las atiende en orden estricto: una petición lenta bloquea a las que están detrás. **HTTP/2** (2015) multiplexa varias peticiones sobre una conexión y comprime los encabezados. **HTTP/3** (2022) reemplaza TCP por QUIC, que corre sobre UDP y elimina el bloqueo de cabecera de línea que persistía en la capa de transporte.

Lo relevante para este módulo es que la semántica no cambió en ninguna de las tres. Los métodos, los códigos y los encabezados de la sección 1.4 son idénticos en las tres versiones. Cambió **cómo se transportan los bytes**, no qué significan.

Y una noticia buena para vos

Que la semántica no haya cambiado en treinta años **no es casualidad**: es la primera decisión de diseño de la sección 1.2 dando frutos.

Lo práctico: **casi todo lo que aprendas de HTTP te va a seguir sirviendo dentro de diez años.** Es de las inversiones más rentables que podés hacer en esta carrera. Las herramientas cambian todo el tiempo; los protocolos, casi nunca.

1.13 — Verificación: el checklist honesto

Siete comprobaciones. **No son ejercicios: son el criterio para saber si el capítulo se entendió.**

- Abrir las herramientas de desarrollo en cualquier sitio, ubicar la petición del documento principal y **nombrar tres encabezados de petición y tres de respuesta**, explicando qué informa cada uno. (1.5)
- Provocar deliberadamente un **404** pidiendo una ruta inexistente, y **distinguir en el panel de red esa respuesta de un fallo de conexión**. (1.4.3)
- Comparar el resultado de **Ctrl+U** con el panel de elementos en un sitio con scripts, y señalar al menos una diferencia. (1.7.1)
- Escribir un documento HTML mínimo con las cinco líneas de cabecera y **explicar por qué está cada una**. (1.8)
- Recorrer con la tecla **Tab** un formulario propio y verificar que todos los campos reciben foco en un orden razonable. (1.9)
- Emitir una petición con `curl -i` y localizar en la salida **la línea de estado, los encabezados y el cuerpo**. (1.11)
- Tomar una fila cualquiera de la sección 6 del TPI y **explicar sus cinco columnas** como se hizo en la sección 1.10. (1.10)

1.14 — Los ocho errores frecuentes

Todos tienen algo en común: **en el momento, no parecen errores**. Por eso son frecuentes.

El error	Por qué duele	Sección
Confundir el HTML con el DOM	Es el más extendido y el que más tiempo hace perder. Se manifiesta cuando alguien busca en el código fuente un elemento que agregó por programa, no lo encuentra y concluye que su código no funcionó. El código funcionó: está mirando el lugar equivocado	1.7.1
Tratar todos los 4xx igual	Mandar al usuario al login ante un 403 es el caso típico. Ya está autenticado; lo que falta es permiso. Volver a pedirle la contraseña no cambia nada y lo desconcierta	1.4.3
Suponer que la validación del cliente protege el servidor	Es la regla RN-F04 , y el error se comete casi siempre por optimización: «esto ya lo validé antes». La validación del cliente mejora la experiencia y no aporta ninguna garantía	1.9.2
Reintentar un POST sin clave de idempotencia	Ante una respuesta que no llega, la reacción natural es reintentar. Sin clave, eso duplica el pedido : el usuario recibe dos veces la misma comida y un cargo doble	1.4.2
Omitir el <code><meta charset></code> o ponerlo tarde	Acentos y eñes partidos. Y el diagnóstico es engañoso, porque el archivo se ve bien en el editor : el problema está en cómo lo interpreta el navegador, no en cómo se guardó	1.8
Usar <code><div></code> con manejador de clic en lugar de <code><button></code>	Se ve idéntico y es inutilizable con teclado o lector de pantalla . Suele descubrirse tarde, cuando rehacerlo cuesta mucho más	1.9.1
Poner datos sensibles en la cadena de consulta	Quedan en los logs del servidor, en el historial del navegador y en el Referer de la navegación siguiente	1.6
Creer que HTTPS valida el contenido	HTTPS acredita la identidad del servidor y protege el canal. Un sitio con certificado válido puede mentir con total comodidad	1.12

1.15 — Las actividades, y qué busca cada una

Siete actividades. Debajo de cada una, lo que en realidad quiere que descubras.

1. Reconstruir el recorrido

Elegir un sitio de uso cotidiano, abrir el panel de red y documentar las primeras cinco peticiones: método, ruta, código de estado y tipo de contenido. Indicar cuáles corresponden al **paso 4** de la sección 1.3 y cuáles al **paso 6**.

Qué busca: *que dejes de pensar en «cargar una página» y empieces a ver decenas de peticiones independientes.*

2. Clasificar métodos

Para cada endpoint de la sección 6.8 del TPI —el módulo de pedidos— determinar si el método es **seguro**, si es **idempotente**, y justificar en una línea si esa clasificación es la adecuada para lo que el endpoint hace.

Qué busca: *que descubras algún endpoint donde el método elegido y la semántica no coinciden del todo. Ahí empieza el criterio.*

3. Diagnóstico de códigos

Para cada uno de los códigos **400, 401, 403, 404, 409, 422, 429 y 500**, redactar una situación concreta del dominio del TPI que lo produzca **y qué debería hacer el cliente al recibirlo**.

Qué busca: *la segunda mitad —qué hacer— es la que convierte el catálogo en manejo de errores útil.*

4. Documento semántico

Escribir la estructura HTML de una página de catálogo **usando exclusivamente elementos semánticos, sin ningún <div>**. Verificar el resultado en el panel de accesibilidad y anotar qué rol recibió cada elemento.

Qué busca: *que la restricción te obligue a preguntarte qué significa cada bloque, en vez de envolver todo en cajas neutras.*

5. Comparación DOM / HTML

En un sitio con contenido dinámico, capturar el código fuente original y el DOM actual, y señalar **tres diferencias explicando el origen de cada una**.

Qué busca: *ver con los ojos la distinción de la sección 1.7.1. Después de esto ya no se olvida.*

6. Exploración: peticiones sin navegador

Reproducir con `curl` una petición observada en el panel de red, **incluyendo sus encabezados**, y comparar la respuesta con la que mostró el navegador. Relacionarlo con la ausencia de estado de la sección 1.4.1: **¿por qué la petición funciona igual fuera del navegador?**

Qué busca: *que compruebes que el servidor no sabe ni le importa quién le está hablando. Eso es no tener estado.*

7. Exploración: el costo de la tolerancia

Escribir **deliberadamente** un documento con errores de marcado —etiquetas sin cerrar, anidación cruzada, atributos inventados— y comparar en el panel de elementos el árbol que el navegador

construyó con el que esperabas. Relacionarlo con la tercera decisión de diseño y explicar **qué se ganó y qué se perdió**.

Qué busca: la experiencia de que el navegador te «arregle» el error en silencio. Es exactamente lo que hace en producción cuando el error no es a propósito.

1.16 — Síntesis: las diez frases

1. La web nació para resolver **un problema de recuperación de información distribuida sin coordinación central**. Sus cuatro decisiones de diseño explican todo lo observable en este módulo, **incluidas sus molestias**.
2. Las alternativas técnicamente superiores fracasaron porque exigían coordinación. **La web resignó garantías a cambio de escalar**, y esa es la clase de intercambio que hay que saber identificar en cualquier tecnología.
3. **HTTP no tiene estado**. El servidor no recuerda nada entre peticiones, y por eso escala replicándose. La contracara es que cada petición debe traer consigo lo necesario para reconstruir el contexto: de ahí el token en cada llamada.
4. **La semántica de los métodos no es formalidad**. POST no es idempotente, y de esa propiedad — no de una preferencia del enunciado — sale la exigencia de una clave de idempotencia en el checkout: la regla RN-F07.
5. **Un código de estado es información de diagnóstico**. La familia 4xx dice que la petición estaba mal y reintentarla no sirve; 5xx dice que el servidor falló. Y **no recibir respuesta no es lo mismo que recibir un 500**: en un caso la petición llegó y en el otro no.
6. **El HTML no es el DOM**. El primero es el texto que llegó; el segundo, la estructura que el navegador construyó. Todo el trabajo de los capítulos siguientes ocurre sobre el segundo.
7. **El navegador nunca rechaza marcado inválido: lo recupera en silencio**. Por eso un error de marcado no produce mensaje de error, y por eso ninguna validación del cliente constituye una garantía.
8. **La etiqueta correcta no es decoración**: determina el rol en el árbol de accesibilidad, y con él el comportamiento de teclado, foco y lectores de pantalla. Un `<div>` estilizado como botón obliga a reimplementar a mano todo lo que `<button>` trae gratis.
9. **HTTPS acredita identidad y protege el canal; no valida contenido**. Los encabezados de seguridad agregan una capa que actúa aunque la aplicación tenga errores.
10. **La semántica de HTTP no cambió en treinta años**, pese a tres versiones del transporte. Lo aprendido acá sigue siendo válido mucho después de que las herramientas de moda hayan sido reemplazadas.

1.17 — Qué leer, y en qué orden

El original lista las fuentes en dos párrafos densos. Acá van ordenadas por prioridad real.

Si lees una sola cosa

Kurose y Ross, *Computer Networking: A Top-Down Approach* (8.^a edición, Pearson, 2021), el capítulo de capa de aplicación. Presenta HTTP con el mismo enfoque descendente de este capítulo y **es la referencia más accesible para quien viene de programación y no de redes**.

Si lees tres

- **Grigorik**, *High Performance Browser Networking* (O'Reilly, 2013 — de lectura libre en hpbn.co): el recorrido completo de una petición y las razones de rendimiento detrás de cada versión del protocolo.
- **Garsiel e Irish**, *How Browsers Work* (2011): describe el pipeline de renderizado de la sección 1.7 con más profundidad que cualquier documentación de producto.

- **Berners-Lee**, *Weaving the Web* (Harper, 1999): el relato de su autor documenta de primera mano las decisiones de la sección 1.2 y **las alternativas que se descartaron**.

Las fuentes normativas (para consultar, no para leer de corrido)

- **HTTP**: la **RFC 9110** (*HTTP Semantics*, 2022) reemplazó a la conocida RFC 2616 y define los métodos, los códigos y la clasificación de seguridad e idempotencia de la sección 1.4.2. La acompañan la **9111** (caché) y la **9112** (sintaxis de HTTP/1.1). Las versiones nuevas del transporte: **9113** (HTTP/2) y **9114** (HTTP/3), esta última sobre QUIC (**RFC 9000**). Todas libres en rfc-editor.org.
- **El URI**: **RFC 3986**, la sintaxis de la sección 1.6.
- **Contexto histórico**: la **RFC 1945** documenta HTTP/1.0 tal como se usaba de hecho en 1996, y su lectura muestra con claridad **qué problemas motivaron la versión siguiente**.
- **HTML**: no es una RFC sino un **estándar viviente** del WHATWG, en html.spec.whatwg.org. Sus secciones sobre el algoritmo de parseo documentan las reglas de recuperación de errores de la sección 1.7.1.
- **Accesibilidad**: las **WCAG 2.2** del W3C, y la especificación **WAI-ARIA**, que define el modelo de roles, nombres y estados del árbol de la sección 1.9.

Cierre: las seis cosas que hay que recordar

Si dentro de un mes te acordás de seis frases de todo esto, que sean estas.

LAS SEIS

1. La web **resignó garantías para no necesitar coordinación**. Ante cualquier tecnología, la pregunta correcta no es sólo si funciona: es **qué resignó para funcionar**.
2. El protocolo **no se acuerda de nada**. Por eso el token viaja en cada llamada, y por eso el sistema puede crecer agregando máquinas.
3. **POST no es idempotente**. De ahí sale la clave de idempotencia del checkout — no del enunciado.
4. **4xx sos vos, 5xx soy yo**. Y no recibir nada no es lo mismo que recibir un 500: en un caso hay log del otro lado, en el otro no llegó nunca.
5. **El HTML es el plano; el DOM es la casa**. Ctrl+U te muestra el plano; el panel Elements, la casa.
6. **La etiqueta correcta trae comportamiento gratis**. Un `<div>` disfrazado de botón es una puerta pintada en la pared.

Y una séptima, que no está escrita en el capítulo pero está en todas sus páginas: **cuando algo de la plataforma parece un capricho, buscá la decisión de 1989 que lo originó**. Casi siempre está ahí, y casi siempre tiene sentido. Eso es lo que separa aprender la web de padecerla.